Blastoid API

Concept

The **Blastoid API** is featured in Quick Launch for ReText.

Key Features

- **HTML** Scraper
- File Download with **Rico**
- **JSON** Scraper Results
- **JSOM** Scraper Results

The primary use case is to examine (scrape) **HTML SCRIPT** elements. Though it also supports *any element* with an **ID** value.

The main **Entry Point**, which doubles as an **Accessor Object**, is `blastoid()`. This function accepts a single **ID value** as an argument. It then locates the object and reads its content.

Content is based on the object's **HTML** `nodeName` property.

The **ID** is used as a **filename**. This and the content are passed to the `rico()` method. The overall action is to *export* an **HTML element** as **Plain Text.**

Source Files

- **Morpheus**
  - Owl System
- **LAN Hosts**
  - Omega
  - Tower
  - Legacy

Members

Name	Type	Purpose
`options`	Object	**Blob Options** for `rico`
`jsom`	Method	Scrape Elements as **JSOM**
`json`	Method	Scrape Elements as **JSON**
`mods`	Method	Scrape Elements as **Core Table**
`read`	Method	**Read** Element's **Content**
`rico`	Method	Save **Plain Text** File

File I/O

The core method is `rico()`. **Rico** saves a **Plain Text** file to the local **File System**.

These functions/methods invoke `rico()` to save a file:

- `blastoid()`
- `jsom()`
- `json()`

There is one caveat. The `blastoid()` function ***always*** saves the file. The other two (methods) ***only*** save a file if a **filename** is passed as the *second* argument. Otherwise, the content is read and ***formatted***, then returned as either **JSOM** or **JSON** text.

Scraping

In this context, ***scraping*** is defined as the act of *extracting text* from zero or more **HTML** Elements.

Blastoid requires an **ID** or **List of ID values** in order to locate the target Element(s).

Once located, each Element is passed to the `read()` method in order to extract text.

The final result may take any of these forms:

Method	Return Type	Comments
`blastoid`	Text	`HTML, JS, CSS, etc.`
`mods`	Core Table	`See Below`
`jsom`	JSOM Text	`See Below`
`json`	JSON Text	`See Below`

Content Read

This method extracts text from an HTML element. The property read is determined by the nodeName for the element.

Node Name	Property
INPUT	value
TEXTAREA	value
PRE	innerText
CODE	innerText
VAR	innerText
Other	innerHTML

Core Table

This term is from my personal lexicon. It's how I distinguish this format from other Table Formats. It's nothing more than an Array of Arrays, whose values are all Text Strings.

The first cell in each row contains the Element's ID value. The second cell contains the Element's Content.

Example

`id-editor-1`	// Some JS Content
	alert("Hello!");

`ed-editor-2`	# Some Markdown Content - List Item - List Item - List Item
`id-viewer-1`	Just some plain old text.

Note that there is no schema information. It's implicit in the formatting rules.

JSOM

This is another term from my personal lexicon. JSOM is similar to an old Plain Text file with Form Feed markers. However, the ASCII FF character isn't used. Rather I use a special line to separate "Pages".

The Page Separator line begins and ends with the vertical bar character ("|"). It's exactly 64 characters wide. Between the bracketing vertical bars is a series of 62 dash marks ("-") or, in some cases, tilde ("~").

Example

```
|--------------------------------------------------------------|
| id-editor-1
|--------------------------------------------------------------|
// Some JS Content
alert( "Hello!" );
|--------------------------------------------------------------|
| id-editor-2
|--------------------------------------------------------------|
# Some Markdown Content
   -  List Item
   -  List Item
   -  List Item
|--------------------------------------------------------------|
| id-viewer-1
|--------------------------------------------------------------|
Just some plain old text.
|--------------------------------------------------------------|
```

Note that the example shown above doesn't use the full 64 characters for separator lines. This was a limitation of the tool used to create this document.

JSON

The `json()` method merely *stringifies* a **Core Table**.

Example

```
[
  [ “id-editor-1”, “...” ],
  [ “id-editor-2”, “...” ],
  [ “id-viewer-1”, “...” ]
]
```

Related Links

- This Document
- **Published eBook**
  - Morpheus
  - Omega
- **Quick Links**
  - Omega
- **Morpheus**
  - Home Page
  - Snek
  - Owl System Test
- **LAN Hosts**
  - Omega ReText Notes
- **Project Tools**
  - Project Notes
  - Gemini Chats
- **Related Projects**
  - HUD Editor
  - Chatter
  - Tonto Editor